



Análise Estática Generativa com LLMs: Otimizando a Detecção de Vulnerabilidades no SemGrep via Abordagem Multi-Agente

Caio Xavier da Silva

SENAC — Unidade Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Leonardo Massayuki Takuno
Coorientador: Thyago Conchado Quintas
São Paulo, 2025

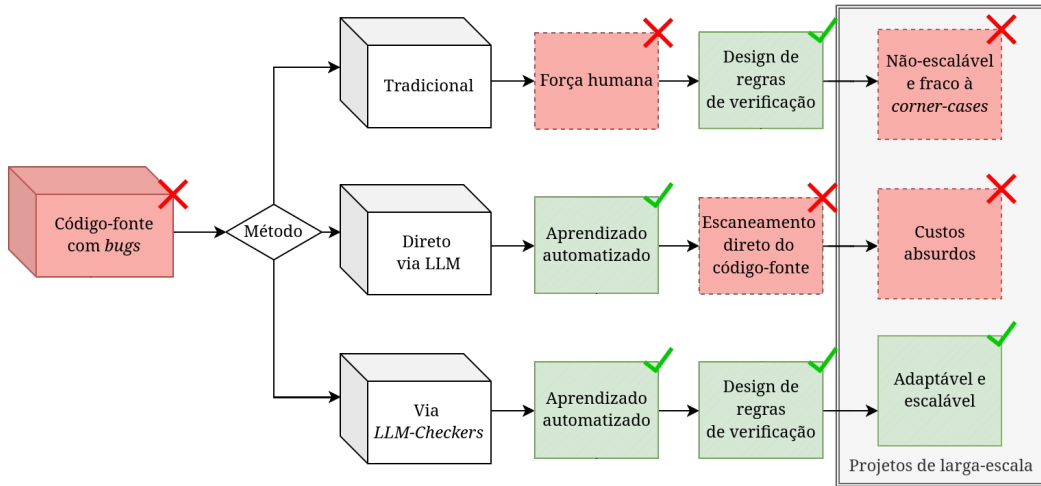
- 1 Introdução
- 2 Referencial Bibliográfico
- 3 Desenvolvimento

- Análise estática e sua utilização. ([HOU et al., 2025](#); [JIANG et al., 2025](#))
- **Limitações da análise estática tradicional:**
 - Escalabilidade e Diversidade de *Bugs*; ([YANG et al., 2025](#))
 - Esforço Humano; ([HOU et al., 2025](#))
 - Falta de Extensibilidade Multilinguagem; ([HOU et al., 2025](#))
 - Falsos Positivos e Falsos Negativos. ([JIANG et al., 2025](#))

- LLMs na análise estática. ([TANG et al., 2021](#); [YANG et al., 2025](#))
- **Limitações desses modelos:** ([TUNSTALL; WERRA; WOLF, 2022](#); [HOU et al., 2025](#))
 - Custos proibitivos (milhões de *tokens*);
 - Restrições práticas (*self-attention*).

- **Multi-Agentes:** ([RUSSELL; NORVIG, 2020](#))
 - Processamento descentralizado e distribuído entre agentes;
 - Cada agente assume um papel restrito e especializado;
 - Reduzir sobrecarga de *tokens*;
 - Maior precisão na geração do artefato final.
- **Analizador estático ideal:** ([YANG et al., 2025](#))
 - Detectar uma diversidade de falhas;
 - Eficientemente processar milhões de linhas de código.
- Motivações (Figura 1). ([YANG et al., 2025](#))

Figura 1: Motivação para *checkers* feitos por LLMs.



Fonte: Adaptado pelo autor para melhor compreensão (YANG et al., 2025)

- LLMs na análise estática. ([HOU et al., 2025](#); [YANG et al., 2025](#); [LI et al., 2026](#))
- **Limites desse método:** ([HOU et al., 2025](#); [YANG et al., 2025](#); [LI et al., 2026](#))
 - Cobertura limitada;
 - Falha em variações semânticas complexas;
 - *corner-cases*.
- Análise estática via regras (SemGrep, Figura 2).

Figura 2: Exemplo de regra criada para detectar uma senha embutida no argumento da função em Python.

rules:

- *id:* rule-id

message: >-

Senha embutida é usada como argumento padrão na função '\$FUNC'. Isso pode ser perigoso se uma senha verdadeira não é atribuída.

Template da mensagem de bug.

languages: [python]

severity: WARNING

Linguagens alvo.

patterns:

- *pattern:* |

def \$FUNC(..., password="...", ...):

...

- *pattern-not:* |

def \$FUNC(..., password="", ...):

...

Ignora funções com senhas vazias.

Fonte: Adaptado para português com finalidade de melhorar compreensão (HOU et al., 2025)

Justificativa formal

Portanto, este trabalho justifica-se pela necessidade de preencher a atual lacuna de eficácia e escalabilidade na detecção de vulnerabilidades. O desenvolvimento de um *framework* iterativo, focado na geração automatizada de regras para o motor do SemGrep, é motivado em mitigar a dependência de engenharia manual e contornar os altos custos da inferência direta por IAs. Ao propor um sistema especializado nas vulnerabilidades críticas (CWEs) da linguagem C, essa pesquisa visa demonstrar que é possível identificar padrões semânticos e *corner-cases* com maior precisão e viabilidade técnica do que os analisadores estáticos tradicionais ou métodos generalistas baseados exclusivamente em LLMs.

Hipótese formal

A hipótese desta pesquisa é a de que a geração automatizada de regras por meio de um *framework* multi-agente baseado em LLMs, integrado ao motor do SemGrep, produzirá taxas de detecção (*Recall* e *F1-Score*) de vulnerabilidades (CWEs) em C estatisticamente superiores e com maior identificação de *corner-cases* quando comparada aos conjuntos de regras predefinidas dos analisadores estáticos tradicionais (como por exemplo *ruleset* tradicional do SemGrep).

Objetivo Geral

O objetivo geral deste trabalho é demonstrar que um *framework* de síntese automatizada de regras, baseando-se em Modelos de Linguagem de Larga Escala e integrado à ferramenta SemGrep, apresenta precisão e *recall* estatisticamente superiores às abordagens estáticas tradicionais na detecção de vulnerabilidades críticas em linguagem C. Busca-se demonstrar que este modelo supera abordagens estáticas tradicionais na identificação precisa de, no mínimo, cinco categorias de vulnerabilidades de software, em inglês *Common Weakness Enumeration* (CWE).

Objetivos específicos

- Determinar os limites de detecção semântica (*corner-cases*) que podem ser superados pela abordagem gerativa de LLMs em relação aos conjuntos de regras predefinidas (*ruleset*) estáticos.
- Estabelecer o ganho quantitativo de desempenho (*Recall*, Precisão e F1-Score) de regras sintetizadas dinamicamente frente aos analisadores tradicionais do estado da arte.
- Determinar a viabilidade técnica e a escalabilidade de inferência da arquitetura proposta utilizando modelos de linguagem de menor escala (com contagem reduzida de parâmetros), visando garantir a viabilidade técnica da execução do *framework* em infraestruturas locais com restrições de custo e *hardware*.

- **Motor de pesquisa:** Google Acadêmico.
- **Bases consultadas:**
 - IEEE Xplore;
 - arXiv;
 - ACM Digital Library.
- **Palavras-chave:** Figura 3.

Figura 3: *Query* utilizada durante a pesquisa de materiais.

```
"static analysis" AND ("LLM" OR "large language model" OR "GPT") AND ("C" OR "C++" OR  
"Linux kernel") AND ("pointer" OR "undefined" OR "memory" OR "value-flow") AND  
("SemGrep" OR "semgrep")
```

Fonte: Elaborado pelo autor (2026)

Critérios de Inclusão

- ① Trabalhos publicados entre 2021 e 2026;
- ② Disponibilidade nos idiomas inglês ou português;
- ③ Relação direta com o tema.

Critérios de Exclusão

- ① Trabalhos publicados antes de 2021;
Exceção de referências clássicas/seminais.
- ② Artigos sem alinhamento direto com análise estática ou geração de regras de código;
- ③ Duplicidade de publicações.

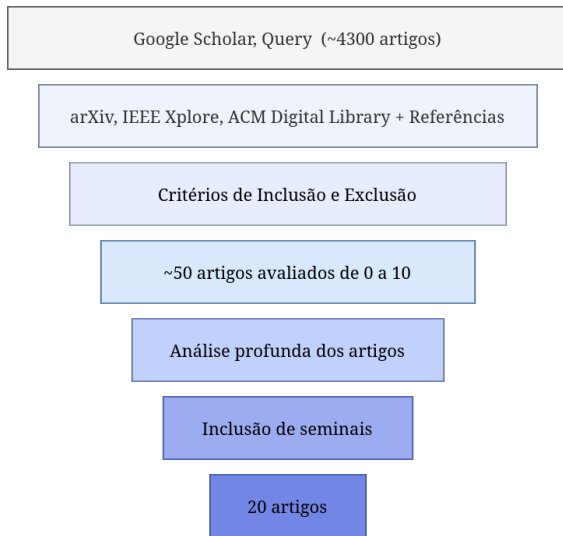
① Primeira fase (triagem):

- *Query* retornou cerca de 4770 resultados.
- Selecionados 47 potencialmente relevantes.

② Segunda fase (afunilamento):

- Leitura de introdução e escopo.
- Classificação de 0 a 10 de acordo com os objetivos deste trabalho.
Avaliação feita a partir de seu resumo, palavras-chave, tecnologias aplicadas.
- Totalizando em 15 trabalhos contemporâneos (6 relevantes e 9 suporte).
- Adicionando 5 trabalhos seminais anteriores a 2021.

Figura 4: Funil de pesquisa.



Fonte: Elaborado pelo autor (2026)

Análise Estática Baseada em Regras

- Ferramentas relevantes: SemGrep e CodeQL. ([JIANG et al., 2025](#); [HOU et al., 2025](#))
- **SemGrep:**
 - Suporta estruturas complexas e verificações básicas de fluxo de dados.
 - Gramática em YAML mimetiza a linguagem alvo.
 - Curva de aprendizado é menor que CodeQL.
 - Converte o código em uma árvore de sintaxe abstrata universal (UAST).
 - Exemplo (Figura 5).

Figura 5: Exemplo de regra simplificada para a detecção de falhas OSCI em Java.

```
1  patterns:
2    - pattern-inside: |
3      $FUNCDECL(..., $USER_DATA, ...) {...}
4    - pattern-either:
5      - pattern: |
6        $PB.command(..., $USER_DATA, ...)
7      - patterns:
8        - pattern-either:
9          - pattern-inside: |
10            $X = $USER_DATA; ...
11          - pattern-inside: |
12            $X = trim($USER_DATA); ...
13        - pattern: $PB.command(..., $X, ...);
```

Fonte: Adaptado de ([JIANG et al., 2025](#))

SemGrep utiliza dois recursos fundamentais:

- ① **Metavariáveis**, auxiliando no fluxo de dados.
- ② **Curinga** (...), permitindo omitir detalhes não relevantes.

Predicados:

- ***patterns***, representando *AND* (E).
- ***pattern-inside***, define o contexto.
- ***pattern-either***, representa *OR* (OU).
- ***pattern-not***, representa *NOT* (NÃO).

Vulnerabilidades de Linguagem

- Vulnerabilidade é um conjunto de condições que permite um invasor (atacante) viole uma política de segurança explícita ou implícita. (??)
- ***Common Weakness Enumeration*** é um dicionário formal desenvolvida pela comunidade que categoriza fraquezas comuns de *software* e *hardware*.
- **Fraquezas (*Weakness*)** é definido como uma condição que um componente de *software*, *firmware*, *hardware* ou serviço pode contribuir para a introdução de uma vulnerabilidade. (??)

CWEs selecionadas

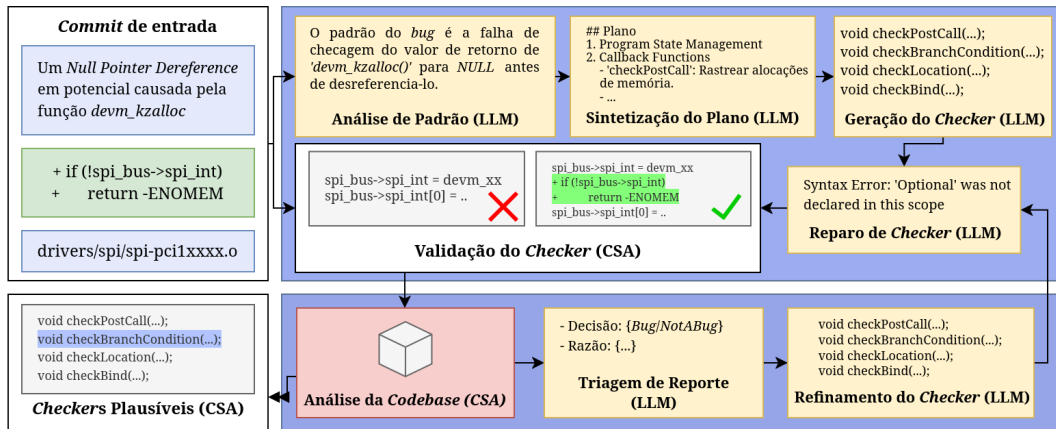
- **CWE-401:** *Missing Release of Memory after Effective Lifetime;*
- **CWE-415:** *Double Free;*
- **CWE-416:** *Use After Free;*
- **CWE-457:** *Use of Uninitialized Variable;*
- **CWE-476:** *Null Pointer Dereference.*

KNighter: Transforming Static Analysis with LLM-Synthesized Checkers (YANG et al., 2025)

- Publicado em SOSP 25 (*ACM SIGOPS 31st Symposium on Operating Systems Principles*), em Seul, República da Coreia.
- KNighter foi o primeiro sistema de geração de analisadores estáticos totalmente automatizado. (YANG et al., 2025)
- Motivado pelas limitações da análise estática tradicional.
- Propõe um *framework* voltado à extração de *commits*.
- *Overview* do trabalho (Figura 6).

- **Motor principal:** *Clang Static Analyser* (CSA).
- Como *commit* de entrada, KNighter retorna como saída um *checker* para *Clang Static Analyser* (CSA). (YANG et al., 2025)
Sendo um verificador válido aquele que identifica o código pré *commit* como *bug* e o código corrigido como correto.
- **KNighter possui duas fases:** (Figura 6)
 - Sintetização de verificadores;
 - Reparo para erros.
- **Diferenças do presente trabalho:**
 - CSA vs. SemGrep;
 - Utiliza somente *commits* para geração de regras.

Figura 6: *Overview do KNighter.*



Fonte: Adaptado para o português para melhor compreensão (YANG et al., 2025)

Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models (HOU et al., 2025)

- Publicado no IEEE International Conference on Software e Quality, Reliability and Security (QRS) em 2025.
- Representa um dos trabalhos que compõem o atual estado da arte em Análise Estática Automatizado até o momento da elaboração desse TCC.
- Motivado para melhorar análise estática para *bugs* contextualizados, lidando com *corner-cases*.
- Capacidade de migrar regras para diferentes linguagens.

- Propõe método de geração de regras a partir de padrões de históricos de código e *few-shot learning* em contexto.
- Suporte para migração de regras existentes para outras linguagens.
- Utilização do SemGrep.
- **Workflow de três estágios:**
 - Geração, a partir de *patches* gera descrições;
 - Migração, a partir das descrições gera regras;
 - Otimização, cria *snippets* para testes de *corner-cases*.

- Validação manual para cada regra.
- **Papel para o presente trabalho:**
 - Aprimoramentos do pioneiro KNighter;
 - Potencial com SemGrep;
 - Referencial para utilização de ferramentas tradicionais para integrações com LLMs.

LLM-based Vulnerability Detection at Project Scale: An Empirical Study (LI et al., 2026)

- Publicado no arXiv em 27 de janeiro de 2026, aguardando revisão em IEEE Transactions on Software Engineering (TSE).
- **Preprint**
- Trabalho mais atual na área de análise estática de vulnerabilidades automatizados via LLMs.
- **Preencher as lacunas:**
 - Efetividade e Complementaridade em projetos de mundo real;
 - Análise abrangente sobre Causas-Raíz de falhas;
 - Escalabilidade e Overhead.

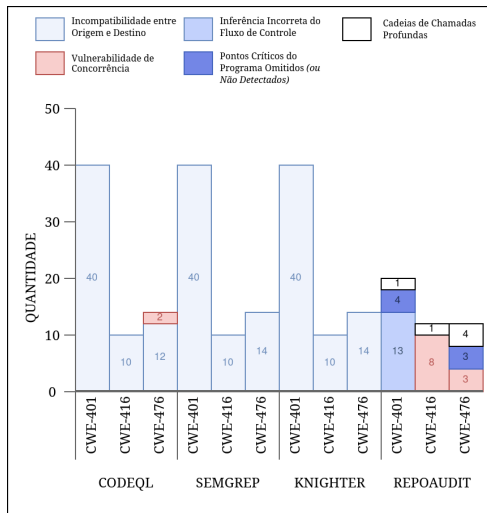
- **Escopo:**

- Estudo empírico;
- 5 projetos baseados em LLMs;
- *Dataset* com 222 vulnerabilidades em C/C++ e Java.

- **Resultados:**

- Cobertura limitada de vulnerabilidades;
Recall médio de 21.09% em C, falta de contexto e definição contínua de variável.
- Ambos métodos tradicionais e baseados em LLMs exibem alta taxa de falsos positivos;
Melhor ferramenta com 85.3% de taxa média.
- Métodos baseados em LLMs continuam computacionalmente caro.
Custar entre cem mil *tokens* à cem milhões de *tokens* por projeto.
Processamento de múltiplas horas para múltiplos dias.

Figura 7: Análise de CWEs (*Memory Leak, Use After Free, Null Pointer Dereference*)



Fonte: Adaptado para o português para melhor compreensão (Li et al., 2026)

- **Source-Sink Mismatch**, definições não correspondem às APIs reais (como `malloc` e `dev_alloc_skb`).
- **Mis-Inferred Control Flow**, falha em entender caminhos de execução, como ramificações aninhadas ou definições `goto`.
- **Concurrency Vulnerability**, falha em acompanhar *threads*.
- **Deep Chains**, vulnerabilidades que exigem a análise de um contexto profundo de chamadas de funções.
- **Missed Critical Program Points**, falha de inferência da LLM ao pular declarações cruciais no meio do código.

Figura 8: Comparação técnica entre os trabalhos relacionados.

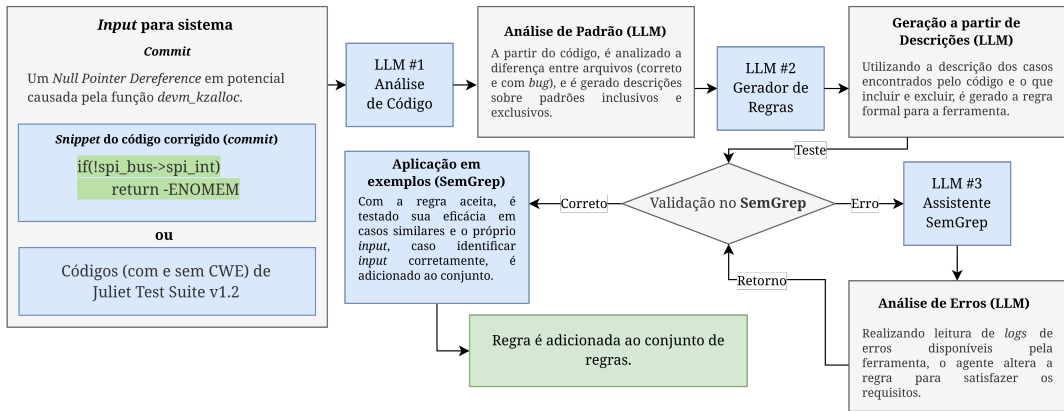
Critério	Yang et al. (2025)	Hou et al. (2025)	Li et al. (2026)	Este trabalho
Foco principal	Geração automatizada de verificadores estáticos a partir de histórico de <i>patches</i> .	Geração, otimização e migração multilinguagem de regras de análise estática.	Estudo empírico comparativo de detectores baseados em LLMs em escala de projeto.	Geração de regras automatizada a partir de histórico de <i>patches</i> e <i>datasets</i> .
Método/ Tecnologia	LLMs + Clang Static Analyser (CSA)	LLMs (<i>few-shot learning</i>) + SemGrep	Avaliação de múltiplos agentes/geradores LLM frente a analisadores tradicionais.	LLMs (<i>few-shot learning</i>) + <i>CWEs datasets</i> + SemGrep
Métrica Principal de Avaliação	Taxa de validade dos <i>checkers</i> e volume de novos <i>bugs</i> detectados.	Validade sintática/ funcional e eficácia na transposição de regras entre linguagens.	<i>Recall</i> , Taxa de Falsos Positivos e custo computacional (tempo/ <i>tokens</i>).	<i>Recall</i> , Taxa de Falsos Positivos e custo computacional (tempo/ <i>tokens</i>).
Limitação / Trabalho Futuro	Acoplamento estrito ao ecossistema C/C++ e geração de falsos positivos diante de lógicas de estado complexas.	Alta taxa de falsos positivos e dificuldade em fluxo de dados.	Taxas de falso alarme insustentáveis decorrentes de raciocínio de fluxo raso e custo de inferência proibitivos.	–

Fonte: Elaborado pelo autor (2026)

- **Método utilizado:** *Design Science Research* (DSR), focado na resolução de problemas.
- Artefato projetado para otimização da análise estática em projetos de larga escala, realizando uma solução computacional avaliável.
- **Proposta:** Extensão Geradora de Regras baseado em LLMs direcionada a Projetos de Larga Escala para a ferramenta de análise estática, SemGrep.
- **Estruturado em quatro fases:**
 - 1 Construção de *dataset*;
 - 2 *Pipeline* de geração de regras;
 - 3 Avaliação de resultados;
 - 4 Aplicação no mundo real.

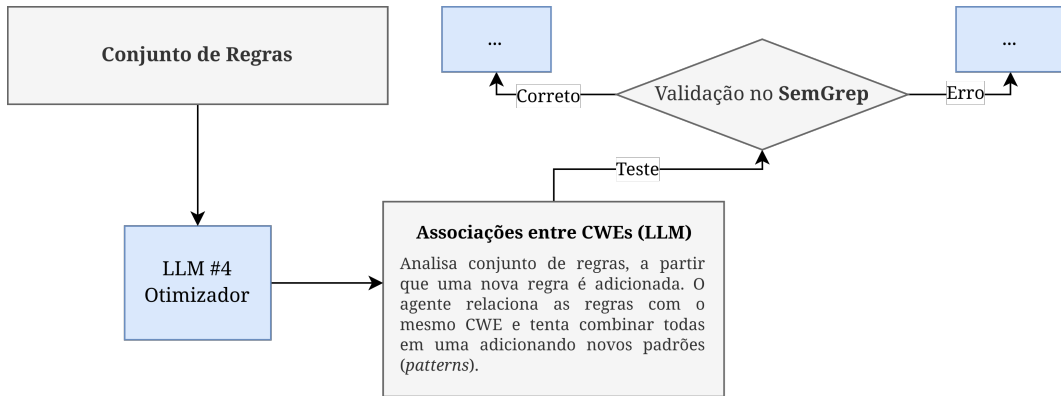
- **Dois módulos independentes:**
 - ① **Módulo de *Dataset***, utilizando o repositório de códigos de ??) e históricos de *commits* de projetos *open-source* ativos;
 - ② **Camada do *Pipeline* (Agentes LLM):**
 - Análise e Abstração (#1);
 - Síntese de Sintaxe (#2);
 - Assistência e Validação (#3);
 - Otimização Contínua e Consolidação de Regras (#4).
- **Métricas de avaliação:**
 - *Precision*, *Recall* e F1-Score;
 - Validação Gramática;
 - Validação Funcional;
 - Validação em mundo real;
 - Custo.

Figura 9: Pipeline para o gerador de regras.



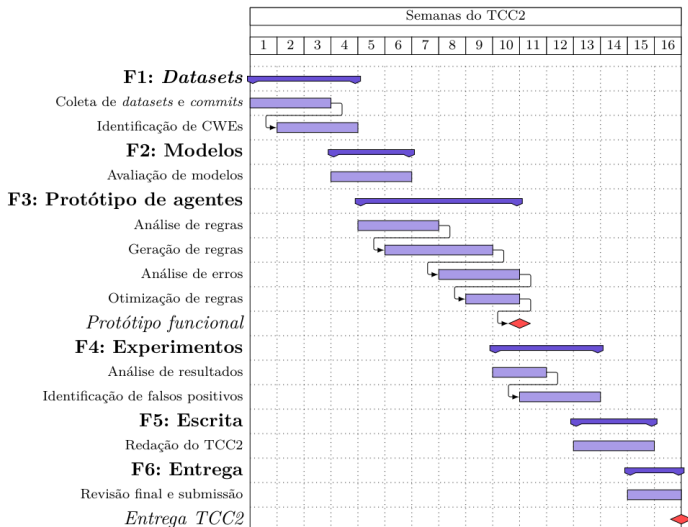
Fonte: Elaborado pelo autor (2026)

Figura 10: Pós-processamento para otimizar regras e abranger *corner-cases*.



Fonte: Elaborado pelo autor (2026)

Figura 11: Cronograma de atividades - Diagrama de Gantt.



Fonte: Elaborado pelo autor (2026)



Obrigado!

Perguntas?

Caio Xavier da Silva
caioxaviercx@gmail.com

 HOU, Z. et al. Generation, migration and optimization of cross-language static-analysis rules based on large language models. In: *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE)*. [s.n.], 2025. Disponível em: <<https://ieeexplore.ieee.org/document/11173453/>>. 3, 4, 7, 8, 18, 26

 JIANG, Z. et al. Fact-aligned and template-constrained static analyzer rule enhancement with llms. In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l.: s.n.], 2025. 3, 18, 19

 LI, F. et al. *LLM-based Vulnerability Detection at Project Scale: An Empirical Study*. 2026. Disponível em: <<https://arxiv.org/pdf/2601.19239>>. 7, 29, 31

 RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Hoboken: Pearson, 2020. ISBN 978-0134610993.

5

 TANG, Y. et al. Grammar-based patches generation for automated program repair. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, 2021. p. 1300–1305. Disponível em: <<https://aclanthology.org/2021.findings-acl.111/>>.

4

 TUNSTALL, L.; WERRA, L. von; WOLF, T. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. Sebastopol, CA: O'Reilly Media, Inc., 2022. ISBN 978-1098103248.

4

 YANG, C. et al. Knighter: Transforming static analysis with llm-synthesized checkers. *arXiv preprint arXiv:2503.09002*, 2025. Disponível em: [3, 4, 5, 6, 7, 23, 24, 25](https://arxiv.org/abs/2503.09002).